

# Works on My Machine, Works in CI

Treating Your Build Toolchain as a Dependency

LUIS CARO CAMPOS

# Works on My Machine, Works in CI

Treating Your Build Toolchain as a Dependency



Luis Caro Campos  
JFrog



# Introduction

# Dependencies

```
requests>=2.25, <3.0.0
urllib3>=1.26.6, <2.1
colorama>=0.4.3, <0.5.0
PyYAML>=6.0, <7.0
patch-ng>=1.18.0, <1.19
fasteners>=0.15
Jinja2>=3.0, <4.0.0
python-dateutil>=2.8.0,
<3
```

requirements.txt



```
[package]
name = "foobar"
version = "1.0.0"

[dependencies]
bitflags = "2.9.4"
rand = "0.9.2"
```

cargo.toml



```
{
  "name": "foobar",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2",
    "nodemon": "^3.0.1"
  }
}
```

package.json



???

??



# Package managers for C++ development

```
{  
  "name": "foobar",  
  "version-string": "1.0.0",  
  "dependencies": [  
    "fmt",  
    "zlib"  
  ]  
}
```

vcpkg.json



vcpkg

```
[requires]  
fmt/10.0.0  
zlib/1.2.13  
  
[generators]  
CMakeDeps  
CMakeToolchain
```

conanfile.txt|py



Conan

```
# Build Instructions  
sudo apt-get install \  
  libfmt-dev \  
  zlib1g-dev  
  
# ...Good Luck!
```



README.md



apt

# Install workflow - it just works!



vcpkg

```
cmake -B build -S . \
  -DCMAKE_TOOLCHAIN_FILE=$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake \
  -DCMAKE_BUILD_TYPE=Release
```



Conan + cmake-conan

```
cmake -B build -S . \
  -DCMAKE_PROJECT_TOP_LEVEL_INCLUDES=path/to/conan_provider.cmake \
  -DCMAKE_BUILD_TYPE=Release
```



Conan + CMake

```
# Step 1: install dependencies and generate CMake integration files
conan install . --build=missing -s build_type=Release

# Step 2: configure and build using the generated preset
cmake --preset conan-release
cmake --build --preset conan-release
```

# Software Bill of Materials and Compliance

## What is an SBOM?



### Inventory

Complete "list of ingredients" detailing components, libraries and dependencies



### Standardized Formats

Machine-readable files structured in standardized file formats like SPDX or CycloneDX.



### Supply Chain Visibility

Maps component relationships and software provenance across the entire development lifecycle.

## Why Companies Use Them



### Regulatory Compliance

Mandated by governments and industry standards across Automotive, Medical, and Aerospace fields.



### Vulnerability Auditing

Core registry for scanning and instantly locating known vulnerabilities (CVEs) in third-party libraries.



### Legal & License Safety

Mitigates compliance risks by tracking open-source licenses.

# Build toolchains

## Developer Reality



### Unmanaged Setup

Developers are left on their own following setup wikis, or relying on IT-lead enterprise device management. Can lead to everyone using slightly different versions of the same toolchain.



### Absent from SBOMs

Toolchains are often omitted from automated SBOM capture, requiring manual intervention to track.

## Traceability Gap



### Embedded Code

Final binaries embed toolchain code directly—including C++ standard library code and pre-compiled runtimes (e.g. libgcc\_s, crt1.o, etc).



### Untracked Dependency

We track application dependencies, yet risk leaving toolchain / system dependencies unmonitored.

# C++ project: where do dependencies come from?



## C++ Package Managers

**Conan / vcpkg, for project dependencies**

Managed via project spec files living alongside source code.

- Declared in project repo (e.g. conanfile.py, vcpkg.json)
- Explicit versions & settings
- Reproducible per-project builds



## External installers

**For build toolchains**

Compilers and core tools installed outside the project setup.

- Visual Studio Installer (Windows)
- App Store / Software Updates (macOS)
- Distro package manager (Linux)



## Distro package managers

**For compiler and system SDK**

Shared system libraries provided by the host environment.

- Common on Linux & \*nix setups
- Managed via distro tools (apt, dnf)

# Typical toolchain installation



## Windows

### Visual Studio Installer

- MSVC C++ Build Tools
- Windows SDK headers and libraries
- IDE integration



## macOS

### App Store and/or Software Updates

Xcode or Command Line Tools

- /Applications/Xcode.app (as installed by App Store)
- /Library/Developer/ (as installed by CommandLineTools)
- AppleClang and system SDK



## Linux

### Distro Package Manager

System package repositories management.

- GCC / Clang packages
- System headers, libraries, and additional tools

- First installation: typically manual
- Updates: may happen automatically

# Toolchain drift

## REALITY

### Version Divergence

Developers **likely** to have slightly different versions of the compiler toolchain or one of its key components

- Slightly different compiler toolchain versions based on installation date
- Automatic system or OS package manager updates - again also based on when last updated
- Different OS distributions and SDK releases

## EXPECTATIONS

### Compatibility guarantees

**Why teams assume drift is harmless**

C++ and compiler vendors are prioritizing compatibility

- C++ language design prioritizes strong backwards compatibility
- Compiler vendors avoid breakages, both with code and with the ABI of the C++ standard library
- Minor compiler patch updates are expected to be risk-free



**It's harmless to update, ... right?**

# Toolchain drift example: MSVC

## CASE STUDY: PACKAGE BREAKAGE

[package] foobar: crashes if built against MSVC 2026 instead of MSVC 2022 #12345

🔗 Open

### Observed behavior:

- affects all versions of the package
- built fine under: **MSVC 2022, MSVC 2026 (19.50)**
- crashes when compiled with: **MSVC 2026 (19.51 update)**



## ROOT CAUSE

### An actual compiler bug

- Started failing when updating MSVC to the most recent version at the time
- Source code remains identical, but binary output crashes observed in some codepaths
- Difficult to diagnose / troubleshoot, not much recourse other than rolling back the update

# Toolchain drift example: Apple Clang

## Upgraded to Sequoia & Xcode 16.0, now build doesn't work

Developer Tools & Services

Xcode

Xcode



kithrup

Created

Sep '24

Replies

3

Boosts

1

Views

1.1k

Participants

3

```
1 /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX15.0.sc
2 'to_chars' is unavailable: introduced in macOS 13.3
3 66 | to_chars_result __r = std::to_chars(__first, __last, __value, __fmt);
```

Which, ok, I can accept that this is true. Except that this worked on Friday, on the same machine running Sonoma and the previous version of Xcode. The project is configured for a deployment target of 12.0, so it should have failed before, but didn't.

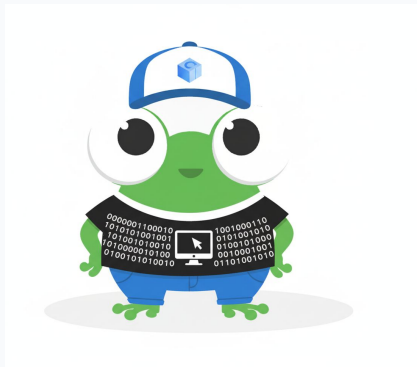
(This is a CMake-generated xcodeproj, but that also should not have been any change.)

↑ Boost



# Toolchain drift at scale

## 1 DEVELOPER



### Manageable

Can probably handle it manually

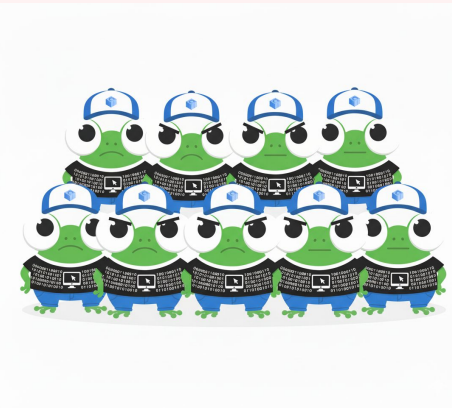
## SMALL TEAM



### Alignment Needed

Can coordinate through direct communication

## LARGE TEAM



### Major Time Sink

Multiple OSs, CI machines, and toolchain variations

# Rolling out a compiler update

This is good — but still doesn't solve the alignment issue. Whenever we move to a newer version, we may have to:



01

## Notify Team

Let developers know about the new toolchain version and coordinated updates.



02

## Update Containers

Edit Dockerfiles, rebuild base images, and push updated containers to registries.



03

## Provision VMs

Log in to install on VMs manually or run provisioning automation tools.

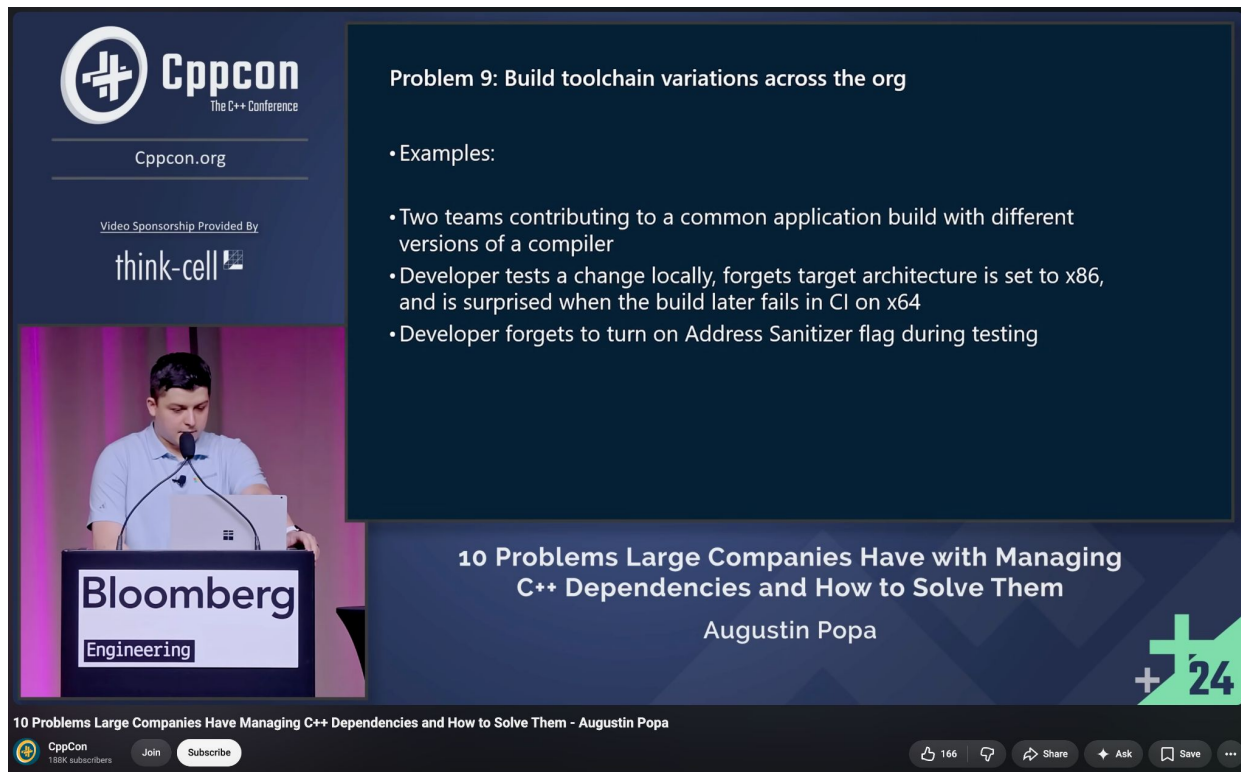


04

## Heterogenous Steps

Maintain potentially different setup steps and configurations for each OS platform.

# Previously on... CppCon



**Cppcon**  
The C++ Conference

Cppcon.org

Video Sponsorship Provided By  
**think-cell**

**Problem 9: Build toolchain variations across the org**

- Examples:
- Two teams contributing to a common application build with different versions of a compiler
- Developer tests a change locally, forgets target architecture is set to x86, and is surprised when the build later fails in CI on x64
- Developer forgets to turn on Address Sanitizer flag during testing

**10 Problems Large Companies Have with Managing C++ Dependencies and How to Solve Them**

Augustin Popa

**Bloomberg**  
Engineering

**+ 24**

10 Problems Large Companies Have Managing C++ Dependencies and How to Solve Them - Augustin Popa

CppCon  
188K subscribers

Join Subscribe

166 | | Share | Ask | Save | ...

# A recurrent topic



r/cpp • 3mo ago  
minshinji

...

## How do you keep toolchain drift across windows, linux, and mac manageable at 30+ engineers?

We're about 30 engineers split across Windows and Linux, with a handful on macOS. All C++, CMake + Ninja as the build system. The compiler situation is where it gets messy. Someone's always on a slightly different version and it surfaces in the strangest ways, usually right before a release.

We've gone back and forth on a few approaches...pinning versions across machines, containerizing the build environment, leaning on build servers and treating local builds as second-class, or some mix. None feel clean. All have tradeoffs, and the tradeoffs shift depending on the platform.

What's held up at a similar scale?

# A recurrent topic (cont'd)



r/cpp • 1mo ago  
TheRavagerSw

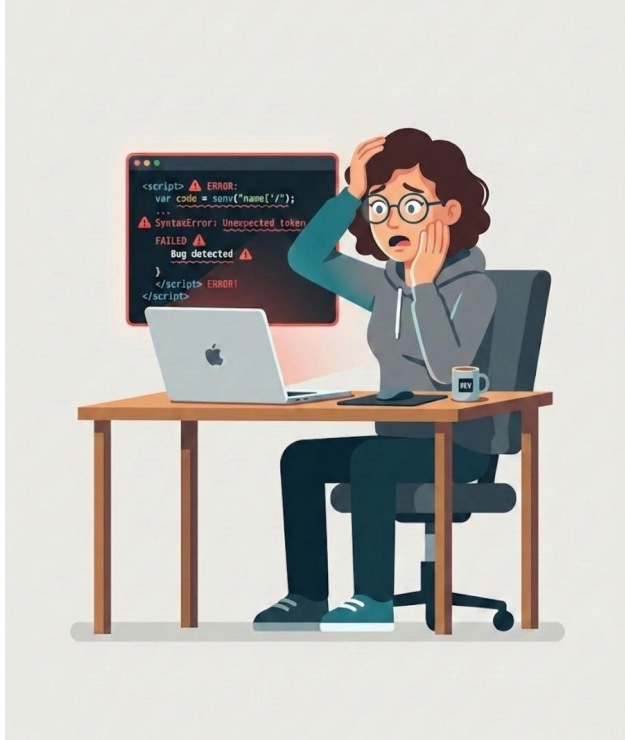
...

## GCC should ship prebuilt binary tarballs like LLVM

It is really annoying, when you wanna test something you can't just download a previous release like llvm, so it is very hard to check if there is a regression.

Also, the default build of c and c++ only, takes like an hour because of triple consecutive builds. So every time you have to lookup to that weird flag on the internet to turn that off.

# Modern Problems



**Claude decided to upgrade the compiler toolchain of a team member -without asking- and broke the build**

**Claude apologized afterwards**

# Developer Productivity

When the primary goal is **productivity** and **saving engineering time** (as opposed to just reproducibility) there are two straightforward solutions to mitigate the time lost to toolchain drift:

01

???

???

02

???

???

# Solution #1: error out early

## If you can get away with it:

in your project or toolchain files... just show a clear error if a developer is running an untested compiler version  
A developer following Readme/Wiki instructions to get the right version may take less time than troubleshooting mysterious bugs

```
if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
    if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS "9.0" OR CMAKE_CXX_COMPILER_VERSION VERSION_GREATER "14.99")
        message(FATAL_ERROR "GCC version ${CMAKE_CXX_COMPILER_VERSION} not supported. Supported: 9.x - 14.x")
    endif()
elseif(CMAKE_CXX_COMPILER_ID STREQUAL "AppleClang")
    if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS "12.0" OR CMAKE_CXX_COMPILER_VERSION VERSION_GREATER "15.99")
        message(FATAL_ERROR "AppleClang version ${CMAKE_CXX_COMPILER_VERSION} not supported. Supported: 12.x - 15.x")
    endif()
elseif(CMAKE_CXX_COMPILER_ID STREQUAL "MSVC")
    if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS "19.20" OR CMAKE_CXX_COMPILER_VERSION VERSION_GREATER "19.99")
        message(FATAL_ERROR "MSVC version ${CMAKE_CXX_COMPILER_VERSION} not supported. Supported: 19.20 - 19.99")
    endif()
else()
    message(FATAL_ERROR "Unsupported compiler: ${CMAKE_CXX_COMPILER_ID}")
endif()
```

# Solution #1: error out early (cont'd)

## AVOID (POTENTIAL FOR UNBOUNDED TIME TASK)

### Follow-up for unexplained toolchain drift error: ✗

- Time spent troubleshooting, debugging, etc.
- May happen right before a release
- May not have actionable solutions if it is a true compiler bug

## PREFERRED (MORE PREDICTABLE)

### Follow-up when a developer gets "untested compiler version error" ✓

- Follow known process (e.g., Wiki, README)
- Time-bounded and quick if familiar with it

# Solution #2: install the compiler elsewhere

## Rule of Thumb:

Don't use a compiler that lives in a folder managed by an external process (Apple Store, macOS system updates, distro system updates, visual studio installer)

### AVOID (MANAGED BY SOME EXTERNAL PROCESS)

**NO** ❌

- `/Applications/Xcode.app`
- `/usr/bin/g++`
- `C:/Program Files/Visual Studio/`

### PREFERRED (ISOLATED & VERSIONED)

**YES** ✅

- `/Applications/Xcode-21.0.0.app`
- `/usr/local/bin/my-gcc15.1-toolchain/bin/g++`
- `???` for `msvc`

**Warning / Caveat:** This still doesn't resolve what the contents of that folder are, how to get it, and how it gets there... but if you have the option, go for it!

# Solution #2: install the compiler elsewhere

Tried and tested!!



The screenshot shows the Compiler Explorer interface for the compiler `arm64 msvc v19.51 VS18.6`. The source code in `example.cpp` is as follows:

```
1 # License: MSVC Proprietary
2 # The use of this compiler is only permitted for internal evaluation
3 # See https://visualstudio.microsoft.com/license-terms/vs2022-g
4 ;ARM64
5
6 |void foo(void)| PROC ; foo
7 ret
8
9 ENDP ; |void foo(void)|, foo
```

The compiler output shows the paths used for various compiler components, with the path `Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\cl.exe` highlighted by a red box:

```
example.cpp
Compiler Passes:
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\cl.exe: Version 19.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\c1.dll: Version 19.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\c1xx.dll: Version 19.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\c2.dll: Version 19.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\c1xx.dll: Version 19.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\link.exe: Version 14.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\mspdb140.dll: Version 14.51.36246.0
Z:\compilers\msvc\14.51.36231-14.51.36246.0\bin\Hostx64\arm64\1033\clui.dll: Version 19.51.36246.0

Compiler returned: 0
```



Running Compiler Explorer in 2026: Sandboxes, Storage, and Strangers' Code

Tuesday September 15, 2026 14:00 - 15:00 MDT

Colorado A

**Thank You**

**(just kidding)**

# **Packaging Build Toolchains**

# Packaging a build toolchain

## Goals

We want be able to install compiler toolchains in arbitrary, relocatable folders, where a specific version is isolated, e.g.:

```
/opt/acme/clang23-linux/
```

```
/opt/acme/gcc15-toolchain
```

```
Z:/compilers/msvc/14.51.36231
```

```
/opt/toolchains/Xcode-16.0.1.app
```

## Challenges

- **Hermetic and self-contained:** can they be used without requiring files in other locations?
- What are the actual contents and **how** do they get there
- Getting your builds to use this toolchain - instead of system defaults

# Packaging a Build Toolchain



01. Linux (+ others)  
**LLVM Clang**



02. Android  
**Android NDK**



03. macOS (and iOS derivatives)  
**Apple Clang**



04. Windows  
**MSVC**



05. Linux (+ others)  
**GCC**

# LLVM Clang on Linux

## Release list

LLVM 23.1.1

LLVM 23.1.0

LLVM 23.1.0-rc3

LLVM 23.1.0-rc2

LLVM 22.1.8

LLVM 22.1.7

LLVM 22.1.6

LLVM 22.1.5

LLVM 22.1.4

LLVM 22.1.3

< Previous Next >

## LLVM 23.1.1

Latest

Compare

github-actions released this 3 days ago llvmorg-23.1.1 6dfe167

### LLVM 23.1.1 Release

- Linux x86\_64: [xz archive](#) (signature), [zstd archive](#) (signature)
- Linux Arm64: [xz archive](#) (signature), [zstd archive](#) (signature)
- macOS Apple Silicon (ARM64): [xz archive](#) (signature), [zstd archive](#) (signature)
- Windows x64 (64-bit): [installer](#) (signature), [xz archive](#) (signature), [zstd archive](#) (signature)
- Windows on Arm (ARM64): [installer](#) (signature), [xz archive](#) (signature), [zstd archive](#) (signature)

Download links for common platforms will appear above once builds have completed, if they are available. Check the full list of release packages at the bottom of this release page if you do not find a link above.

If you do not find a release package for your platform, you may be able to find a community built package on the LLVM Discourse forum thread for this release. Remember that these are built by volunteers and may not always be available. If you rely on a platform or configuration that is not one of the defaults, we suggest you use the binaries that your platform provides, or build your own release packages.

**Straightforward** because there's already a binary distribution (community maintained)

We should be able to:

 **download**

 **unpack**

 **use**

<https://github.com/llvm/llvm-project/releases>

# LLVM Clang on Linux - Experiment



## 01. CONTAINER ENVIRONMENT

### Barebones Docker Image (Ubuntu 26.04)

- CMake
- Ninja and GNU Make



## 02. EXTRACT

### Download and Unzip Toolchain

`/toolchains/LLVM-23.1.1-Linux-X64/`



## 03. ENVIRONMENT VARIABLES

### Set CC & CXX

`CC=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang`  
`CXX=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++`



## 04. VERIFY BUILD

### Sample CMake Project

Hello-world `CMakeLists.txt` that fetches `fmt` via `FetchContent`.

# LLVM Clang on Linux - First attempt

```
$ CC=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang CXX=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ cmake -S . -B build
```

```
-- The CXX compiler identification is Clang 23.1.1
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - failed
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ - broken
CMake Error at /usr/share/cmake-4.2/Modules/CMakeTestCXXCompiler.cmake:73 (message):
  The C++ compiler
```

```
    "/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++"
```

```
is not able to compile a simple test program.
```

```
[...]
```

```
/usr/bin/cmake -E cmake_link_script CMakeFiles/cmTC_db628.dir/link.txt --verbose=1
clang++: error: unable to execute command: posix_spawn failed: No such file or directory
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
```

# LLVM Clang on Linux - First attempt

```
$ CC=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang CXX=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ cmake -S . -B build
```

```
-- The CXX compiler identification is Clang 23.1.1
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - failed
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ - broken
CMake Error at /usr/share/cmake-4.2/Modules/CMakeTestCXXCompiler.cmake:73 (message):
  The C++ compiler
```

```
"/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++"
```

```
is not able to compile a simple test program.
```

⚠ Can't locate the "ld" executable

```
[...]
```

```
/usr/bin/cmake -E cmake_link_script CMakeFiles/cmTC_db628.dir/link.txt --verbose=1
clang++: error: unable to execute command: posix_spawn failed: No such file or directory
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
```

# LLVM Clang on Linux - Second attempt

**CMake Option:** `-DCMAKE_EXE_LINKER_FLAGS="-fuse-ld=lld"`

Instructs CMake to use `ld.lld` (LLVM linker) instead of default system linker

```
CC=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang CXX=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ cmake -S . -B build \
-DCMAKE_EXE_LINKER_FLAGS="-fuse-ld=lld"

-- The CXX compiler identification is Clang 23.1.1
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - failed
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ - broken
CMake Error at /usr/share/cmake-4.2/Modules/CMakeTestCXXCompiler.cmake:73 (message):
  The C++ compiler

    "/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++"

is not able to compile a simple test program.

[ ... ]
Linking CXX executable cmTC_91909
/usr/bin/cmake -E cmake_link_script CMakeFiles/cmTC_91909.dir/link.txt --verbose=1
/toolchains/LLVM-23.1.1-Linux-X64/bin/ld.lld: error while loading shared libraries: libcui18n.so.70: cannot open shared
object file: No such file or directory
[ ... ]
```

## ⚠ Runtime constraints

Bypassed initial `ld` error, but launching `ld.lld` fails immediately.

**Missing Shared Library:**  
`libcui18n.so.70`

## OS Compatibility

- Ubuntu 26.04 — Missing
- Ubuntu 24.04 — Missing
- Ubuntu 22.04 — Works (Fallback)

# LLVM Clang on Linux - Third attempt

**CMake Option:** `-DCMAKE_EXE_LINKER_FLAGS="-fuse-ld=lld"` (Instructs CMake to use `ld.lld` instead of default system linker)

**Runtime constraint:** run inside Ubuntu 22.04

```
CC=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang CXX=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ cmake -S . -B build \
-DCMAKE_EXE_LINKER_FLAGS="-fuse-ld=lld"
-- The CXX compiler identification is Clang 23.1.1
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - failed
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++
-- Check for working CXX compiler: /toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ - broken
CMake Error at /usr/share/cmake-3.22/Modules/CMakeTestCXXCompiler.cmake:62 (message):
  The C++ compiler

    "/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++"

is not able to compile a simple test program.

[...]

Linking CXX executable cmTC_79b2d
/usr/bin/cmake -E cmake_link_script CMakeFiles/cmTC_79b2d.dir/link.txt --verbose=1
/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++ -fuse-ld=lld CMakeFiles/cmTC_79b2d.dir/testCXXCompiler.cxx.o -o cmTC_79b2d
ld.lld: error: cannot open Srt1.o: No such file or directory
ld.lld: error: cannot open crt1.o: No such file or directory
ld.lld: error: cannot open crtbeginS.o: No such file or directory
ld.lld: error: unable to find library -lstdc++
ld.lld: error: unable to find library -lm
ld.lld: error: unable to find library -lgcc_s
ld.lld: error: unable to find library -lgcc
ld.lld: error: unable to find library -lc
ld.lld: error: unable to find library -lgcc_s
ld.lld: error: unable to find library -lgcc
[...]
```

## ⚠ Missing toolchain components

Those libraries the linker is unable to find, are:

- GNU libstdc++
- glibc:
  - `-lm`
  - `-c`
- GCC runtime
  - `-lgcc`

# LLVM Clang on Linux - Two choices

## OPTION 1

### Install a GCC toolchain on the system

Install a complete system + GCC toolchain that Clang can automatically locate when it is running - that means Clang will perform system introspection.

👎 The toolchain is no longer self contained, and those components are just as liable to change ...

## OPTION 2

### Pure LLVM Components

Tell Clang to use LLVM-provided things where possible:

```
-stdlib=libc++ -rtlib=compiler-rt
```

ld.lld: error: unable to find library -lm

ld.lld: error: unable to find library -lc

ld.lld: error: cannot open crtn.o: No such file or directory

We STILL need some system headers and link objects.. So we go back to Path 1 👉

# Clang on Linux - roundup so far

Whilst pre-built binaries are provided, for Linux it is far from self-contained or hermetic:

## TOOLING

### GNU Binutils

Requires system-provided GNU binutils by default for assembly and linking steps

Unless we tell the tools to use the llvm prefixed ones

`ld, as, objcopy, ...`

## INCLUDES

### System Headers

Requires external header files:

- C++ standard headers  
`<vector>` - From GCC unless we tell it to use `libc++`
- C library headers `"limits.h"`
- Other system headers: Linux, POSIX, networking

## DEPENDENCIES

### Linking Libraries

Requires additional external linking libraries:

- GCC runtime (`gcc, gcc_s`)
- C library (`libc.so`)
- C++ library (`libstdc++.so`) - unless we tell it to use `libc++.so`

# The `--sysroot` flag

Tells Clang (or gcc) to treat a specific directory as the logical root (`/`) for headers and libraries.

## MECHANISM

### Path Prefix Substitution

The compiler's internal search logic remains identical—it simply prepends the sysroot directory to default search paths.

#### SEARCH PATH TRANSFORMATION

```
/usr/include → /sysroot/usr/include  
/usr/lib → /sysroot/usr/lib  
/lib → /sysroot/lib
```

## A MORE HERMETIC BUILD

### What it does NOT change

- **Host Compiler Binary:** The compiler executable itself still executes on the host system.
- **Runtime Dynamic Linker:** Only affects link-time finding; does not change dynamic linker paths on the target.
- **Explicit Absolute Paths:** Flags like `-I/path` or `-L/path` bypass sysroot. Use `-isysroot` or relative flags instead.

# Putting together a (barebones) Linux sysroot

Creating a self-contained toolchain by redirecting default search paths to a controlled folder.

## TOOLCHAIN GOAL & CONCEPT

We want the build tools (`compiler`, `linker`) to redirect their default search to a directory we control, bundled directly alongside the compiler.

## DEPENDENCIES

### Linking Libraries

Binary artifacts required at link-time and runtime:

- C Library: `libc.so`, startup objects (`crt1.o`)
- C++ Standard Runtime: `libstdc++.so` or `libc++.so`
- Compiler Runtime: GCC helper routines (`libgcc_s.so`)

## INCLUDES

### System Headers

Header files defining platform interfaces:

- C Standard Headers: `"limits.h"`, `"stdio.h"`
- C++ Standard Headers: `<vector>`, `<iostream>`
- OS & POSIX Headers: Linux kernel & system interfaces

# Putting together a (barebones) Linux sysroot

Extracting missing headers and libraries using Docker and package management.

## AI TO THE RESCUE

### Mini sysroot from Ubuntu packages

- **LLM Prompting:** In the interest of brevity, I asked an LLM for a script template.
- **Package Isolation:** Missing headers and libraries belong to very specific distro packages.
- **Container Workflow:** Install in a throwaway Docker container, query file lists, and copy directly to the sysroot.

## ⚡ NOTE

There's other (and likely better) ways of putting together a sysroot - this is only a minimal example.

```
make_sysroot.sh

mkdir -p sysroot

docker run --rm -v $PWD/sysroot:/out ubuntu:22.04 bash -c "
  apt-get update -qq && apt-get install -y \
    libc6-dev \
    libstdc++-12-dev \
    libgcc-12-dev \
    linux-libc-dev && \

  copy_pkg() {
    dpkg -L $1 | while read f; do
      [ -f $f ] || continue
      mkdir -p /out/$(dirname $f)
      cp -d $f /out/$f
    done
  } && \

  copy_pkg libc6 && \
  copy_pkg libc6-dev && \
  copy_pkg libgcc-s1 && \
  copy_pkg libstdc++-12-dev && \
  copy_pkg libgcc-12-dev && \
  copy_pkg linux-libc-dev
"
```

# LLVM Clang and a custom sysroot



```
CC=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang
CXX=/toolchains/LLVM-23.1.1-Linux-X64/bin/clang++

cmake -S . -B build \
  -DCMAKE_EXE_LINKER_FLAGS="-fuse-ld=lld --sysroot=/toolchains/sysroot" \
  -DCMAKE_SHARED_LINKER_FLAGS="-fuse-ld=lld --sysroot=/toolchains/sysroot" \
  -DCMAKE_CXX_FLAGS="--sysroot=/toolchains/sysroot"

[...]
```

```
[ 57%] Building CXX object CMakeFiles/hello_world.dir/hello_world.cpp.o
[ 71%] Linking CXX executable hello_world
[ 71%] Built target hello_world
```



## RESULT

**Success! Now it works!** 

The custom sysroot binds cleanly and binary targets build as intended.



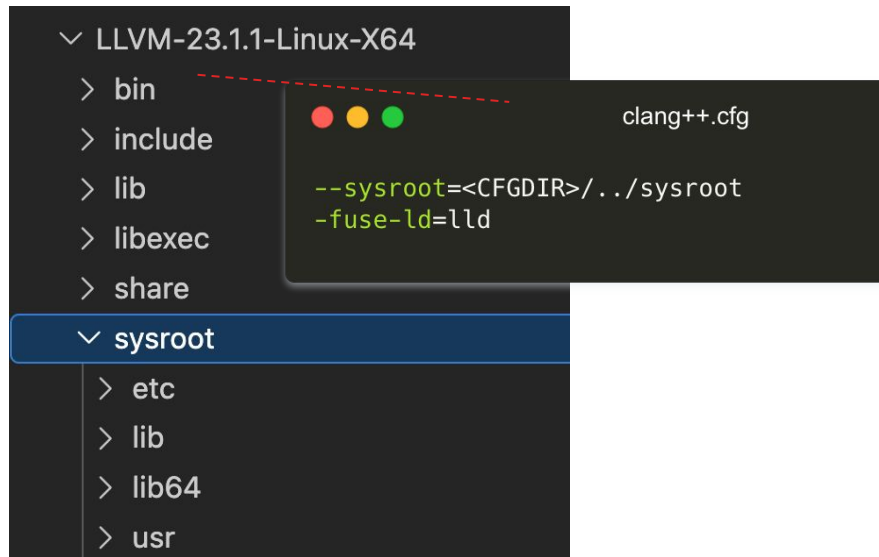
## REALITY CHECK

**...but configuring CMake is a handful now**

Juggling long `-DCMAKE_*_FLAGS` across every invocation gets verbose fast.

# LLVM Clang on Linux - self-contained bundle

Packaging a standalone toolchain with integrated sysroot and implicit configuration.



llvm-clang-23.1.1-linux-x86\_64.tar.gz

## 01 / BUNDLE

### Bundle the sysroot next to clang

- Place the custom `sysroot` directory directly alongside `bin/`, `include/`, and `lib/`.
- Ensures complete self-containment across Linux environments.

## 02 / WIRE CLANG TO PASS THE FLAGS AUTOMATICALLY

### Implicit Flag Invocations

- Add `clang.cfg` and `clang++.cfg` to `bin/`.
- Automatically passes `--sysroot=<CFGDIR>/../sysroot` and `-fuse-ld=lld` without verbose CMake flags.

# Anatomy of a Toolchain

## Core Compiler Toolchain

**Translation Pipeline** (source → .o files → executable)

### Compiler

gcc, clang, g++

### Assembler

as, llvm-as → .o

### Linker

ld, lld, mold, ...

### Dynamic Linker (runtime)

ld-linux.so

## Linking Tools

### Archiver

ar, ranlib → .a

### CRT Startup

crt1.o, crt1.o

## Libraries & Runtime

### C Library

glibc, musl

### C++ Library

libstdc++, libc++.h

## System Headers

### C Headers

stdio.h, stdlib.h

### C++ Headers

<vector>, <iostream>

### Kernel Headers

linux/, asm/

### Built-ins

stdint.h, stddef.h

## Binary Utilities

### Inspection

objdump, nm, readelf

### Manipulation

strip, strings, addr2line

### Size Reporting

size - section sizes

## Optional or External

### Debugging, Profiling & Sanitizers

#### Debugger

gdb, lldb + DWARF

#### Coverage & Profiling

gcov, gprof, llvm-cov

#### Sanitizers

AddressSanitizer, ThreadSanitizer, UndefinedBehaviorSanitizer

### Build System & Utilities

#### Make / Ninja

drive invocations

#### CMake / Meson

generate build files

#### Pkg-config, ccache ...

library discovery & compiler caching

# Android NDK

r27d

[Changelog](#)

```
android {  
    ndkVersion "27.3.13750724"  
}
```



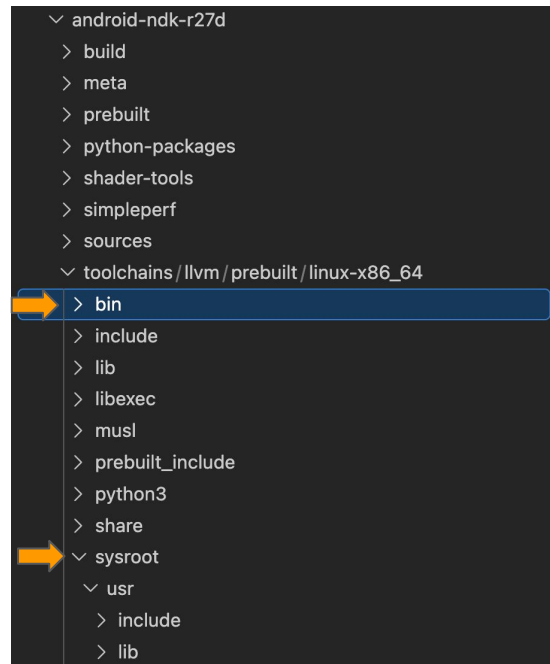
| Platform | Package                                      | Size (Bytes) | SHA1 Checksum                            |
|----------|----------------------------------------------|--------------|------------------------------------------|
| Windows  | <a href="#">android-ndk-r27d-windows.zip</a> | 781506724    | 56607cbccd3642d4a1991f6bb3114a00f884f426 |
| macOS    | <a href="#">android-ndk-r27d-darwin.dmg</a>  | 924374593    | 80f11292080fab4e869799f1d23caa88dcf3c709 |
| Linux    | <a href="#">android-ndk-r27d-linux.zip</a>   | 663956036    | 22105e410cf29afcf163760cc95522b9fb981121 |

## OVERVIEW

### Build Native C++ Code

Use the Android NDK to build native C++ code for Android. Comes in a self-contained, relocatable bundle.

It also has symlinks for `ld` → `ld.lld` - to avoid the need for extra flags



# Android NDK - CMake



```
export NDK=/path/to/android-ndk-r27d
```

```
cmake -S . -B build-android \
```

```
-DCMAKE_TOOLCHAIN_FILE=$NDK/build/cmake/android.toolchain.cmake \
```

```
-DANDROID_ABI=arm64-v8a \
```

```
-DANDROID_PLATFORM=android-24 \
```

```
-DCMAKE_BUILD_TYPE=Release
```

## CONFIGURATION

### We need to specify:

- A CMake toolchain file
- Variables for the architecture and API level

# Android NDK - other build systems

```
$ # Check out the source.
git clone https://github.com/glennrp/libpng -b v1.6.37
cd libpng
$ # Only choose one of these, depending on your build machine...
export TOOLCHAIN=$NDK/toolchains/llvm/prebuilt/darwin-x86_64
export TOOLCHAIN=$NDK/toolchains/llvm/prebuilt/linux-x86_64
$ # Only choose one of these, depending on your device...
export TARGET=aarch64-linux-android
export TARGET=armv7a-linux-androideabi
export TARGET=i686-linux-android
export TARGET=x86_64-linux-android
$ # Set this to your minSdkVersion.
export API=21
$ # Configure and build.
export AR=$TOOLCHAIN/bin/llvm-ar
export CC="$TOOLCHAIN/bin/clang --target=$TARGET$API"
export AS=$CC
export CXX="$TOOLCHAIN/bin/clang++ --target=$TARGET$API"
export LD=$TOOLCHAIN/bin/ld
export RANLIB=$TOOLCHAIN/bin/llvm-ranlib
export STRIP=$TOOLCHAIN/bin/llvm-strip
./configure --host $TARGET
make
```

 [https://developer.android.com/ndk/guides/other\\_build\\_systems](https://developer.android.com/ndk/guides/other_build_systems)

# Android NDK - Build system integration

## CMake Options

- **Built-in CMake support:** Points to NDK location and configures variables.
- **NDK toolchain:** Uses official toolchain file.

### Key CMake Variables:

- `-DCMAKE_TOOLCHAIN_FILE`
- `-DANDROID_ABI`
- `-DANDROID_PLATFORM`

## Autotools & Custom Systems

- Requires setting up specific environment variables manually.

### Compiler & Toolchain Vars:

- `CC CXX AR AS LD RANLIB STRIP`

### Target & Path Vars:

- `TOOLCHAIN TARGET API`

Summary: Your mileage may vary



android-ndk-r27d-linux.zip

# Apple Clang



## Xcode.app

Full IDE Distribution

### Installation Path:

`/Applications/Xcode.app`

### Installation Source:

Installed by App Store

- **Bundled SDKs:** Full sysroots for macOS and other iOS
- **Hermetic Architecture:** Compiler, toolchain, and headers fully packaged together



## Command Line Tools

Without Xcode IDE

### Installation Path:

`/Library/Developer/CommandLineTools`

### Installation Source:

Installed and managed by Software Updates

- **Lightweight Setup:** Lightweight alternative without full Xcode IDE
- **Hermetic:** Compiler, toolchain, and headers fully packaged together

# Apple Clang - Specific versions

Problem:

- Both App Store and Software Updates will always try to update to the latest version - leads to toolchain drift

Possible solution:

- Download specific Xcode versions from Apple Developer Program
- Can be unzipped in any arbitrary location

# Apple Clang - Hermetic toolchains

## System Integration



Easier & more sophisticated

### System-protected locations:

`/usr/bin/clang++, etc`

Default toolchain executables are isolated and managed seamlessly by macOS.

## Redirection



Dynamic Environment Switch

Can be redirected to an arbitrary Xcode installation (or Command Line Tools) via:

`DEVELOPER_DIR`

Allows painless switching between multiple side-by-side toolchains.

## Reproducibility



Hermetic Build Guarantee

For added full reproducibility, explicitly set variables:

`SDKROOT`

`MACOSX_DEPLOYMENT_TARGET`

 Documentation & details: [manpages for xcode-select](#)

# MSVC — Default Installation

## Installation details

- Visual Studio core editor
- ▼ Desktop development with C++
  - ▼ Included
    - ✓ C++ core desktop features
  - ▼ Optional
    - ✓ MSVC Build Tools for x64/x86 (Latest)
    - ✓ MSVC Build Tools for ARM64/ARM64EC (L...
    - ✓ C++ ATL for x64/x86 (Latest MSVC)
    - ✓ C++ ATL for ARM64/ARM64EC (Latest MS...
    - ✓ Image and 3D model editors
    - ✓ C++ Build Insights
    - ✓ Just-In-Time debugger
    - ✓ C++ profiling tools
    - ✓ C++ CMake tools for Windows
    - ✓ Test Adapter for Boost.Test
    - ✓ Test Adapter for Google Test
    - ✓ IntelliCode
    - ✓ MSVC AddressSanitizer



## Default Installer Behavior

### Automated Component Selection

By default, Visual Studio Installer installs the **"latest"** MSVC build tools available at the time of installation.

# MSVC — Side-by-Side Toolchains

Visual Studio Community 2026 — September 2026 Feature Update (18.10.0)

Individual components   Language packs   Installation



143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.41-17.11)(C  
143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.42-17.12)(C  
143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.43-17.13)(C  
143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.44-17.14)  
143 - VS 2022 C++ x64/x86 build tools (v14.33-17.3) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.34-17.4)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.35-17.5) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.36-17.6)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.37-17.7)(Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.38-17.8) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.39-17.9)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.40-17.10) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.41-17.11)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.42-17.12)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.43-17.13)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.44-17.14)



## Side-by-Side Toolchains

Installing Specific & Older Releases

Visual Studio permits side-by-side installation of **older toolset releases** alongside the default latest compiler.

# MSVC — Side-by-Side Toolchains

Visual Studio Community 2026 — September 2026 Feature Update (18.10.0)

Individual components   Language packs   Installation



143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.41-17.11)(C  
143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.42-17.12)(C  
143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.43-17.13)(C  
143 - VS 2022 C++ ARM64/ARM64EC Spectre-mitigated libs (v14.44-17.14)  
143 - VS 2022 C++ x64/x86 build tools (v14.33-17.3) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.34-17.4)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.35-17.5) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.36-17.6)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.37-17.7)(Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.38-17.8) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.39-17.9)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.40-17.10) (Out of support)  
143 - VS 2022 C++ x64/x86 build tools (v14.41-17.11)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.42-17.12)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.43-17.13)(Out of Support)  
143 - VS 2022 C++ x64/x86 build tools (v14.44-17.14)

## VCVARS scripts

Need to pass the `-vcvars_ver=14.38` argument

## CMake

When configuring the project, pass:

```
-G "Visual Studio 17 2022"  
-T v143,version=14.28.29910
```

## MSBuild

Properties → Configuration → Advanced → MSVC  
Toolset Version

# Aligned MSVC versions - some challenges

- Typically developer lead process, but Microsoft offers options
  - Policies for enterprise deployments
  - Custom network installers
  - Fixed version bootstrappers
  - \*.vsconfig files for installation configurations
- Choosing the toolset can still be tricky/manual depending on the tool
- All of the above still leave some global state, e.g.
  - So that vswhere.exe can locate installations
  - Registry Keys

# MSVC relocatable bundles



Hermetic install of MSVC/SDK from the CLI

 Zig ★ 368 🔗 11



PortableMSVC is a command-line tool that automates downloading, caching, extracting, and managing fully portable Microsoft C/C++ toolchains (MSVC and Windows SDK) on Windows—no Visual Studio installation required.

 Python ★ 28 🔗 1

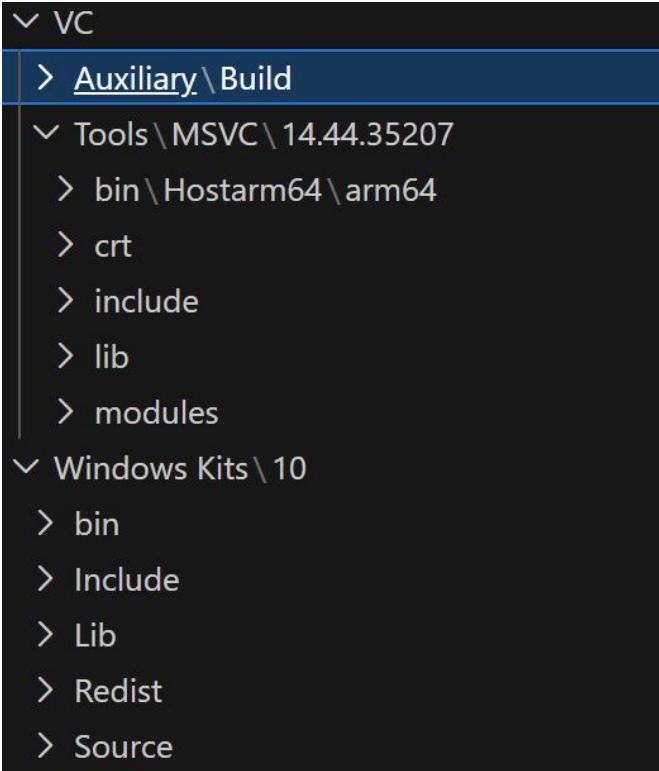


Last active 2 days ago • Report abuse

 **Code**  Revisions 37 ★ Stars 440 🔗 Forks 58  Embed

Download MSVC compiler/linker & Windows SDK without installing full Visual Studio

# MSVC - relocatable bundles



## Mainly:

- The build tools for the specific version (executables)
- Includes and libraries
- Windows SDK

## In order to use it:

- Environment variables that would otherwise be set up by VCVARS (PATH, INCLUDE, LIB, LIBPATH)
- Limited to CMake with Ninja, and anything that works with vcvars

# Recap so far

## LLVM Clang prebuilt binaries

- Just the compiler and some LLVM-flavoured binutils
- Requires **"bring your own system SDK"** (Applies to all platforms)
- Not really self-contained

## Android, AppleClang, MSVC

- **Mostly self-contained (no overlap with the system)**
- SDK is installed as part of the toolchain, tools look there first (and not in global system folders)
- **Linker (Build) vs Runtime Difference:**
  - **Android:** Strictly cross-building (stub libraries)
  - **macOS:** .tbd files inside SDK
  - **Windows:** .lib / .dll split

# GCC on Linux

## Each build of GCC is unique and identified by:

- Version of GCC itself
- Target system (e.g. Linux, Windows mingw, macOS, ...) and CPU architecture
- Version of binutils
- Version of glibc
- Version of Linux headers

## A complete toolchain is an “open ended” thing:

- Looks for headers in **/usr/include** and libraries in **/lib**, **/usr/lib**, **/usr/local**
- Toolchain drift is not strictly limited to just the compiler

# GCC on Linux — Build Decision Tree

## 1. Build Host vs. Target System

Run built binaries on the same system where you build?

YES

**Constraint:** binutils, gcc, glibc & kernel versions are strictly limited to host system

Need to support multiple Linux distros/flavours?

YES

### Target Build

Choose versions matching "target" system; build GCC

NO

### Exact Match

Choose exact versions matching target Linux distro

## 2. GCC Runtime Portability

Will GCC itself need to execute on multiple distros?

### OPTION A: Oldest Baseline

Build GCC on the OS with the **oldest versions** of any runtime dependencies required by GCC itself

OR

### OPTION B: Static Linkage

Use **statically linked dependencies** to prevent dynamic linker failures across heterogeneous glibc environments

## 3. Header & Library Isolation

Look for includes & libraries in global absolute paths?

YES

### Match System Layout

Ensure GCC matches layout variance across target Linux distros:

`/usr/lib` vs. `/usr/lib64` vs. `/usr/lib/x86_64-linux-gnu`

NO

### Self-Contained Sysroot

Build GCC configured with sysroot isolation:

`--with-sysroot`

Place all target headers and libraries inside sysroot

# GCC on Linux — Build Decision Tree



# Linux GCC



## GCC should ship prebuilt binary tarballs like LLVM

It is really annoying, when you wanna test something you can't just download a previous release like llvm, so it is very hard to check if there is a regression.

Also, the default build of c and c++ only, takes like an hour because of triple consecutive builds. So every time you have to lookup to that weird flag on the internet to turn that off.

### If Self-Contained

**No runtime guarantees:** Binaries may fail to run on target systems due to strict version dependencies on **glibc**, **binutils**, and **libstdc++**.

### If Not Self-Contained

**SDK dependency:** Suffers the same problem as Clang binaries — you still need to bring your own SDK, which will likely involve a system-level GCC anyway.

# Isolating GCC: build it from source as a cross-compiler

## 1. Crosstool-ng Setup

Tried & tested toolchain builder:

- Does not look in filesystem root, uses its own sysroot
- Full choice over versions of glibc, binutils, etc.
- Statically link dependencies so GCC runs anywhere

## 2. Effective Benefits

Aligns with Android, AppleClang, MSVC:

- **Self-contained** toolchain environment
- **Separation:** libraries used for linking only, not runtime
- High reproducibility & lower system conflict risk
- Identical compiler across distros/Docker images

## 3. Trade-offs

Added configuration complexity:

- A bit unorthodox when NOT strictly cross-compiling
- Where build systems add `-rpath`, now explicitly need `-rpath-link`
- Anything you would expect to be a "system library" (e.g., libX11, OpenGL, etc.) needs to be pre-shipped in the sysroot

# GCC: Our Current Solution in Conan Center

## Baseline Selection Strategy

For a target GCC version (e.g., GCC 13):

- List mainstream Linux distros & versions with the target GCC 13 runtime ([libstdc++.so.32](#))
- Identify **lowest glibc version** across distros in the list above
- Select lowest required Linux kernel version for kernel headers

## Key Goals & Architecture

Portability and independence across Linux environments:

- **Binary Compatibility:** Binaries built with GCC 13 run on any distro that ships the GCC 13 runtime (or higher)
- **Broad Compiler Support:** Runs across broad Linux flavours without anchoring CI to old Docker/VM instances
- **Decoupled Toolchain:** Isolated from host system libraries to handle distinct filesystem layouts (`/usr/lib`) and varying user library versions

# Recap



`llvm-clang-23.1.1-linux-x86_64.tar.gz`

requires a sysroot for linux



`android-ndk-r27d-linux.zip`

downloaded as is from Android Developer



`Xcode-26.0.1.xip`

specific version downloaded from Apple Developer Program



`msvc-19.44-x64.zip`

downloaded with alternative deployers  
(\*DO\* check your EULAs!)



`my-gcc-13.4-tooclahain-linux-x86_64.tar.gz`

built with crosstool-ng, with an own sysroot

**Putting it all together**

# Toolchain bundles

We now have the key ingredients



01

## Self-Contained Toolchains

A set of compressed folders containing complete build toolchains (compiler, tools, headers, and link libraries).



02

## Build Tool Integration

Knowledge of how to instruct build systems (CMake, Autotools, etc.) to use our toolchain from any specific, arbitrary location.



03

## Side-by-Side support

Ability to install multiple versions side by side on the same machine, simply by deploying each version into a uniquely named folder.

# "Conan install"



vcpkg



Conan + cmake-conan



Conan + CMake



```
cmake -B build -S . \  
-DCMAKE_TOOLCHAIN_FILE=$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake \  
-DCMAKE_BUILD_TYPE=Release
```

```
cmake -B build -S . \  
-DCMAKE_PROJECT_TOP_LEVEL_INCLUDES=path/to/conan_provider.cmake \  
-DCMAKE_BUILD_TYPE=Release
```

```
# Step 1: install dependencies and generate CMake integration files  
conan install . --build=missing -s build_type=Release  
  
# Step 2: configure and build using the generated preset  
cmake --preset conan-release  
cmake --build --preset conan-release
```

# Conan Install

```
conan install [conanfile.py] [--profile:host default] [--profile:build default] \
-s build_type=Release --build=missing
```



## conanfile.py

Not the best place to specify a dependency on a pinned version of a compiler.

For example, you wouldn't have `zlib` declare that it must be built with `gcc13` when it supports everything.



## --profile:host=default

Defines the platform we build for (and the compiler we use for that)

Configures target `"os"`, `"arch"`, and compiler settings. Defaults to `"default"` if omitted.

# Conan profiles

## **[settings]**

```
os=Macos  
arch=armv8  
compiler=apple-clang  
compiler.version=17  
compiler.libcxx=libc++  
compiler.cppstd=gnu17  
build_type=Release
```



macOS

## **[settings]**

```
os=Windows  
arch=x86_64  
compiler=msvc  
compiler.version=194  
compiler.cppstd=14  
compiler.runtime=dynamic  
compiler.runtime_type=Release  
build_type=Release
```



Windows

## **[settings]**

```
arch=x86_64  
build_type=Release  
compiler=gcc  
compiler.cppstd=gnu17  
compiler.libcxx=libstdc++11  
compiler.version=13  
os=Linux
```



Linux

# Conan profiles - toolchains

## [settings]

```
os=Macos
arch=armv8
compiler=apple-clang
compiler.version=17
compiler.libcxx=libc++
compiler.cppstd=gnu17
build_type=Release
```

## [tool\_requires]

```
xcode/26.0.1
```



macOS

## [settings]

```
os=Windows
arch=x86_64
compiler=msvc
compiler.version=194
compiler.cppstd=14
compiler.runtime=dynamic
compiler.runtime_type=Release
build_type=Release
```

## [tool\_requires]

```
msvc/19.44
```



Windows

## [settings]

```
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=13
os=Linux
```

## [tool\_requires]

```
gcc/13.1.0 or clang/23.1.1
```



Linux

# Example: LLVM Clang for Linux with custom sysroot

```
class ClangConan(ConanFile):
    name = "clang"
    version = "23.1.1"
    settings = "os", "arch"

    def build(self):
        download(self, LLVM_URL, "llvm-23.1.1.tar.xz")
        download(self, SYSROOT_URL, "sysroot.tar.gz")

    def package(self):
        unzip(self, "llvm.tar.xz", self.package_folder, strip_root=True)
        unzip(self, "sysroot.tar.gz", f"{self.package_folder}/sysroot", strip_root=True)

    def package_info(self):
        bin = f"{self.package_folder}/bin"
        sysroot = f"{self.package_folder}/sysroot"
        target = "aarch64-linux-gnu"

        self.buildenv_info.prepend_path("PATH", bin)
        self.conf_info.define("tools.build:compiler_executables",
                              {"c": f"{bin}/clang", "cpp": f"{bin}/clang++"})

        flags = ["-fuse-ld=lld", f"--sysroot={sysroot}", f"--target={target}"]
        self.conf_info.append("tools.build:cflags", flags)
        self.conf_info.append("tools.build:cxxflags", flags)
        self.conf_info.append("tools.build:exelinkflags", flags)
        self.conf_info.append("tools.build:sharedlinkflags", flags)
```

Download and unpack to package folder

Flags and configuration to propagate to build systems

# Example: LLVM Clang for Linux with custom sysroot



linux-clang

```
[settings]
arch=armv8
build_type=Release
compiler=clang
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=23
os=Linux

[conf]
tools.build:add_rpath_link=True

[tool_requires]
clang/23.1.1
```

# Demo: LLVM Clang for Linux with custom sysroot

```
linux-clang

$ conan install --require="spdlog/[*]" -o "*:shared=True" -pr linux-clang --build=missing

===== Computing dependency graph =====
Graph root
  cli
Requirements
  fmt/12.1.0#6baf0fb8351783472b94ee6e36232391 - Cache
  spdlog/1.17.0#bcbaaf7147bda6ad24ffbd1ac3d7142c - Cache
Build requirements
  clang/23.1.1#aadf6cd2465f1d1ce5c9e2ba3fc3ca02 - Cache
Resolved version ranges
  spdlog/[*]: spdlog/1.17.0
```

# Demo: LLVM Clang for Linux with custom sysroot

===== Input profiles =====

Profile host:

```
[settings]
arch=armv8
build_type=Release
compiler=clang
compiler.cppstd=gnu20
compiler.libcxx=libstdc++11
compiler.version=23
os=Linux
[options]
*:shared=True
[tool_requires]
*: clang/23.1.1
[conf]
tools.build:add_rpath_link=True
```

Profile build:

```
[settings]
arch=armv8
build_type=Release
compiler=apple-clang
compiler.cppstd=gnu17
compiler.libcxx=libc++
compiler.version=21
os=Macos
[conf]
```

It's a cross-build from macOS!

- The “host” profile would be the same on Linux, what differs is the “build” profile that describes “the machine doing the build”
- Conan exposes enough information to the “clang” recipe:
  - The platform we are targeting (e.g. Linux-aarch64)
  - The platform the compiler itself is running on (in this case, macOS)

# Benefits and downsides

## Benefits

- All developers need to do is have a line in the Conan profiles specifying the compiler version
- Straightforward cross-building support, isolation from build machine
- Same configuration and same command everywhere: Developer machines, CI, Docker images
- Can be cached in Artifactory and served to developers centrally

## Downsides

- Toolchain drift now becomes "conan profile" drift - but we have tools for this :)
- For proprietary toolchains: check your EULAs!
- May have IDE limitations

# Conan Toolchains



conan-io / conan-toolchains

## conan-toolchains

SDKs, compilers & cross-compilers for the Conan ecosystem

 [github.com/conan-io/conan-toolchains](https://github.com/conan-io/conan-toolchains)

```
[settings]
arch=x86_64
build_type=Release
compiler=gcc
compiler.cppstd=gnu17
compiler.libcxx=libstdc++11
compiler.version=13
os=Linux
```

```
[tool_requires]
gcc/13.1.0 or clang/23.1.1
```

In coming weeks:

- Sysroots for Linux
- Re-package LLVM clang
- GCC for Linux
- Proofs of concept for Xcode, MSVC

# Conan Toolchains - cont'd



conan-io / cuda-conan

## cuda-conan

Conan recipes, integration examples, and tutorials for the NVIDIA CUDA Toolkit and CUDA-X GPU accelerated libraries

---

 [github.com/conan-io/cuda-conan](https://github.com/conan-io/cuda-conan)

Similar concept, but for CUDA Toolkit - express this as a dependency and cover native and cross-building workflows

**Thank you**

**Any questions?**

# Works on My Machine, Works in CI

Treating Your Build Toolchain as a Dependency



Luis Caro Campos  
JFrog

